Stack Traces

We've just learned about how function/method calls stack up: put simply, if we have three functions `a()`, `b()`, and `c()`, and each calls the next, this forms a call stack. In well-structured code you'll always have functions calling other functions, objects using other objects… the alternative is a heinous mess I don't want to even contemplate. This is relevant to exception handling because the `catch` block handling the exception does not necessarily exist in the function doing the `throw`(ing) – in fact it's rarely the case.

Call chains (aka the *call stack* or just *stack*) can be debugged with a technique called *bracktracing*: you trace your way backwards from the point of error to (hopefully) find the cause. There are also a couple of useful utility functions in PHP for generating traces manually, but first let's look at how to `throw` and `catch` in different places. We'll make three functions and set them up to call each other, and do a few other exception-related things while we're at it:

```
function a()
{
    try {
        b();
    } finally {
        print "Uncaught in A..." . PHP_EOL;
    }
}
function b()
{
    try {
        c();
    } catch (Exception $e) {
        print "Caught in B..." . PHP_EOL;
        throw $e;
    }
}
function c()
{
    throw new Exception('Thrown in C...');
}
```